



PROYECTO INTERMODULAR 1 DAM

TRAINCORE

SAMUEL REYES, PABLO PALENZUELA Y ALEJANDRO MULLOR



ÍNDICE

BLOQUE 1: PRIMER TRIMESTRE – PLANIFICACIÓN Y DISEÑO DEL PROYECTO	2
1. OBJETIVO DEL PROYECTO Y METODOLOGÍA	2
2. ANEXO DE PLANTILLAS OFICIALES DE INICIO	3
PLANTILLA 1: CONTRATO DE EQUIPO	3
PLANTILLA 2: FICHA DE IDEA DE PROYECTO.....	4
PLANTILLA 3: DOCUMENTO INICIAL DEL PROYECTO (SECCIONES MÍNIMAS)	5
3. GESTIÓN DE RIESGOS Y PIVOTAJE DE PRODUCTO.....	6
BLOQUE II: SEGUNDO TRIMESTRE – DESARROLLO E IMPLEMENTACIÓN TÉCNICA	6
1. CREACIÓN FÍSICA DE LA BD	6
2. NORMALIZACIÓN Y DATOS DE PRUEBA	7
3. DESARROLLO DEL CRUD	7
4. CONEXIÓN APP <-> BBDD Y SEGURIDAD	8
5. FORMULARIOS Y PANTALLAS FUNCIONALES	8
6. TESTING DEL CRUD Y CONTROL DE INCIDENCIAS	9
7. FUNCIONALIDADES ADICIONALES	9
8. DOCUMENTACIÓN TÉCNICA Y ARQUITECTURA	9
9. REVISIÓN INTERNA Y CRUZADA ENTRE EQUIPOS	9
10. ENTREGA PARCIAL 2 Y DEMOSTRACIÓN	10
BLOQUE III: TERCER TRIMESTRE – OPTIMIZACIÓN, CALIDAD Y CIERRE FINAL.....	11
SEMANA 21: MEJORAS DE INTERFAZ Y USABILIDAD.....	11
SEMANA 22: OPTIMIZACIÓN DE CONSULTAS Y RENDIMIENTO.....	11
SEMANA 23: INTEGRACIÓN FINAL DE FUNCIONALIDADES	12
SEMANA 24: PRUEBAS DE USUARIO Y CALIDAD	12
SEMANAS 25 Y 26: REDACCIÓN DE LA MEMORIA FINAL Y REVISIÓN DE LA MEMORIA.....	14
SEMANAS 27 Y 28: PREPARACIÓN DE LA PRESENTACIÓN ORAL Y ENSAYO DE LA DEFENSA.....	14
SEMANAS 29 Y 30: PRESENTACIÓN Y DEFENSA PÚBLICA Y EVALUACIÓN FINAL Y CIERRE.....	15

BLOQUE 1: PRIMER TRIMESTRE – PLANIFICACIÓN Y DISEÑO DEL PROYECTO

1. OBJETIVO DEL PROYECTO Y METODOLOGÍA

En este documento se constituye la memoria técnica y funcional de la aplicación que en un principio era Dublinners que ahora tiene nombre TrainCore. Todo esto se ha desarrollado mediante roles específicos que se asignaron al inicio del curso para que sea más balanceado y que haya un trabajo en equipo:

- **Coordinación:** Responsable de la organización del calendario, la moderación de las sesiones semanales y la formalización de actas y acuerdos de equipo.
- **Programación:** Encargado del diseño técnico, codificación de la lógica de negocio y validación de la arquitectura y patrones de diseño.
- **Documentación:** Responsable de mantener al día los repositorios de información, manuales técnicos y memorias colaborativas.
- **Testing:** Diseñador y ejecutor de planes de pruebas, aseguramiento de la calidad del software (QA) y control del registro de incidencias.

Para la implementación de la aplicación se hizo uso de estas herramientas:

- **GitHub:** Para subir todas las cosas que tengamos dentro del repositorio y que todos podamos manejarlo de manera remota.

- **Google Drive:** Herramienta para guardar la documentación y cada uno pudiera subir las cosas que haya realizado.
- **HTML:** Lenguaje de programación para darle formato y un “esqueleto” a la aplicación y darle la lógica a esta misma.
- **JavaScript:** Herramienta para darle funcionalidad a la app como por ejemplo colores, etc...
- **Node.js (SQL):** Entorno para la ejecución del servidor de la aplicación para la implementación de la base de datos.

2. ANEXO DE PLANTILLAS OFICIALES DE INICIO

PLANTILLA 1: CONTRATO DE EQUIPO

- **DENOMINACIÓN DEL EQUIPO:** (TrainCore)
- **MIEMBROS Y DISTRIBUCIÓN DE ROLES:**
 - Coordinación:** Pablo Palenzuela y Samuel Reyes
 - Programación:** Alejandro Mullor y Pablo Palenzuela
 - Documentación:** Samuel Reyes
 - Testing:** Alejandro Mullor
- **NORMAS INTERNAS DE FUNCIONAMIENTO:**
 1. Toda modificación en la reasignación de tareas debe registrarse en un acta formalizada al equipo
 2. Todos deben informar lo que han realizado en cuanto cambios o tareas realizadas durante la semana
 3. Todos los avances se suben a GitHub al repositorio y cada cosa realizada se informa en la memoria

- **HORARIO ESTIPULADO:** Todos los miércoles de 19:00 – 21:00

PLANTILLA 2: FICHA DE IDEA DE PROYECTO

- **TITULO DEL PROYECTO:** TrainCore
- **DESCRIPCIÓN BREVE:** Aplicación orientada a la gestión integral de la salud física, permitiendo el registro, seguimiento y estructuración de entrenamientos, rutinas e indicadores de rendimiento deportivo contando calorías.
- **PROBLEMAS QUE RESUELVE:** Herramienta que permita tener rutinas, calorías quemadas e incluso en un futuro una dieta personalizada. Que controlen la progresión de cargas y la facilidad de realizar sus rutinas dependiendo de lo que necesiten.
- **USUARIO OBJETIVO:** Atletas que necesiten realizar fuerza, gente que le gusta el deporte y va al gimnasio. Incluso gente que quiera hacer rutina en casa y quiera realizar ejercicio.
- **OBJETIVOS SMART:**
 1. **Específico y medible:** Diseñar e implementar un esquema relacional con tablas normalizadas para usuarios, control de sesiones e historial de rutinas
 2. **Alcanzable:** Programar un módulo funcional de operaciones CRUD completo y endpoints de backend en Node.js que procesen peticiones HTTP antes de concluir el segundo ciclo.
 3. **Relevante y Acotado:** Validar el rendimiento de la persistencia local y el pulido de la interfaz visual aplicando criterios UX/UI en las semanas de consolidación del proyecto.
- **FUNCIONALIDADES MÍNIMAS (MVP):** Autenticación segura de perfiles de usuario, catálogo modularizado de ejercicios físicos,

creador/editor interactivo de rutinas semanales y un generador personalizado según el grupo muscular.

- **RIESGOS INICIALES IDENTIFICADOS:** Complejidad en el acoplamiento de bases de datos tradicionales en ordenadores locales externos.

PLANTILLA 3: DOCUMENTO INICIAL DEL PROYECTO (SECCIONES MÍNIMAS)

1. **Introducción y Contexto:** Justificación del desarrollo de la herramienta en el marco de la digitalización del entrenamiento de fuerza.
2. **Alcance y Objetivos del Proyecto:** Delimitación de las fronteras funcionales del backend local y la interfaz adaptada a un entorno web nativo responsivo.
3. **Requisitos Funcionales:** Detalle de acciones como que el atleta podrá personalizar sus propias rutinas.
4. **Requisitos No Funcionales:** Requisitos de seguridad (cifrado de credenciales mediante hashes criptográficos) y rendimiento (tiempos de respuesta inferiores a 20ms).
5. **Dependencias y Limitaciones:** Restricciones técnicas basadas en el stack. Node.js nativo y Bases de Datos autónomas.
6. **Riesgos Identificados y Mitigación:** Estrategias proactivas de control de versiones en Git y planes de contingencia ante fallos del servidor.
7. **Traza Preliminar:** Matriz cruzada que asocia unívocamente cada requisito funcional con el componente lógico que le da cobertura en la interfaz HTML.

3. GESTIÓN DE RIESGOS Y PIVOTAJE DE PRODUCTO

Durante la fase final del primer trimestre, el equipo experimentó una reevaluación del proyecto. Inicialmente, el proyecto se estaba realizando en la herramienta Figma. Tras esto el docente para que fuera de manera más profesional y funcional se migro el proyecto desde cero a un entorno web haciendo uso de HTML, JavaScript

BLOQUE II: SEGUNDO TRIMESTRE – DESARROLLO E IMPLEMENTACIÓN TÉCNICA

CREACIÓN FÍSICA DE LA BD

Tras el pivotaje al cambio de figma al entorno web de JavaScript y HTML para garantizar la portabilidad absoluta sin necesidad de servidores externos pesados, se implemento lo Base de Datos en SQLite, que fue almacenada localmente. Se hizo la creación de 3 tablas nucleares:

Usuarios: Almacena datos únicos como nombre, correo electrónico y credenciales cifradas

Logins: Registra la auditoría interna de cada inicio de sesión exitoso con marcas de tiempo

Rutinas: Almacena las planificaciones físicas, detallando el nombre del entrenamiento, nivel, duración y vinculación al creador

1. NORMALIZACIÓN Y DATOS DE PRUEBA

Se sometieron las 3 tablas creadas a una revisión de normalización para asegurar las tres primeras formas normales. Se garantizó la atomicidad de campos como el id o el email, se eliminaron dependencias parciales mediante la definición explícita de claves primarias y se quitaron las dependencias transitivas. Para verificar la integridad referencial del modelo antes de conectar la interfaz, se programó la parte inicial de un set de datos de prueba como usuarios simulados todo esto mediante la función de inicialización del motor.

2. DESARROLLO DEL CRUD

Se implementó de manera íntegra las operaciones fundamentales de CRUD en el backend, todo esto tras configurar un servidor HTTP estructurado orientado a endpoints que interactúa directamente en el búfer binario:

Alta (Create): Endpoint que recibe los datos del formulario, comprueba la unicidad del email y realiza la inserción a la tabla usuarios.

Consulta (Read): Endpoint que valida las credenciales de entrada y lee de forma estructurada las rutinas de la base de datos para probar el frontend

Persistencia inmediata: Se integró el módulo del sistema de archivos para que cada operación de escritura que se haga al instante se guarde directamente, evitando la pérdida de información entre reinicios del servidor

3. CONEXIÓN APP <-> BBDD Y SEGURIDAD

Se conectó la página web con el servidor usando el protocolo HTTP para intercambiar información. La web se comunica con el backend de Node.js enviando y recibiendo datos en formato JSON mediante la función fetch de JavaScript. Para mejorar la seguridad, se utilizó la librería crypto de Node.js para proteger las contraseñas. Estas nunca se guardan ni se envían en texto plano, en su lugar, el servidor las convierte en un código cifrado mediante el algoritmo SHA-256 y compara esos códigos durante el inicio de sesión.

4. FORMULARIOS Y PANTALLAS FUNCIONALES

A partir de la migración de figma al entorno web se trasladaron también las vistas a código estructurado real dentro del archivo .html. Esta es la interfaz del usuario que se diseñó bajo una arquitectura compacta de un dispositivo móvil mediante un contenedor centralizado que optimiza el espacio de trabajo. Se crearon diferentes entornos visuales:

Vista de autenticación: Formularios limpios donde se inicia sesión y registro con validación inmediata de campos.

Panel principal: Muestra el perfil del deportista, resúmenes estadísticos y un mapa de calor interactivo que grafica la consistencia de los entrenamientos.

Módulo de creación de rutinas: Formulario con selectores dinámicos de dificultad, duración y enfoque muscular para registrar entrenamientos personalizados.

5. TESTING DEL CRUD Y CONTROL DE INCIDENCIAS

El tester ejecutó planes de prueba específicos sobre los endpoints creados principalmente los de registro e inicio de sesión todo esto utilizando herramientas de simulación de peticiones. Se testearon también escenarios límite como la inserción de correos duplicados, asimismo, para lograr interceptar excepciones y configurar respuestas de error HTTP para evitar caídas imprevistas.

6. FUNCIONALIDADES ADICIONALES

Se mejoró la plataforma web agregando funciones interactivas. También se implementó un sistema de filtros para que los usuarios puedan buscar entrenamientos más fácilmente y se automatizó la visualización de rutinas en el feed según el grupo muscular seleccionado.

7. DOCUMENTACIÓN TÉCNICA Y ARQUITECTURA

Se elaboró el dossier de documentación técnica del proyecto, detallando la arquitectura desacoplada Cliente-Servidor. Se documentaron de forma exhaustiva los scripts SQL de inicialización, el diagrama de flujo de las peticiones fetch, la descripción técnica de los métodos de hashing en server.js y el manual de dependencias de producción reflejado en el archivo package.json.

8. REVISIÓN INTERNA Y CRUZADA ENTRE EQUIPOS

El proyecto fue sometido a una auditoría de evaluación cruzada con otros equipos de desarrollo del aula. Se revisó la viabilidad técnica del despliegue en local a través del comando `node server.js`, la robustez de las validaciones del backend y la velocidad de carga de

la interfaz HTML, recopilando feedback cualitativo para corregir fallos menores en la respuesta visual de los formularios

9. ENTREGA PARCIAL 2 Y DEMOSTRACIÓN

Se procedió al cierre de la fase de desarrollo mediante la congelación de la versión estable del código en el repositorio de GitHub. Se preparó una simulación de demostración técnica local levantando el servidor en el puerto 3001 y ejecutando el flujo completo de la aplicación para certificar que el software estaba listo para ser evaluado.

BLOQUE III: TERCER TRIMESTRE – OPTIMIZACIÓN, CALIDAD Y CIERRE FINAL

SEMANA 21: MEJORAS DE INTERFAZ Y USABILIDAD

Se realizó una revisión completa de la interfaz del HTML. Durante la revisión se evidenció que las distintas pantallas de la aplicación funcionaban mediante clases de CSS dinámicas controladas desde JavaScript.

PROBLEMA IDENTIFICADO: Se identificó que los usuarios podían presentar fatiga visual si las tarjetas de los ejercicios y los botones no tenían un orden dentro del contenedor principal de la aplicación

MEJORAS IMPLEMENTADAS: Para mejorar la experiencia y para que no hubiera problemas en cuanto a la fatiga, se reorganizó y además se cambió la paleta de colores donde se hizo uso de colores oscuros para el fondo y un color más llamativo como es el naranja para destacar acciones importantes como por ejemplo los botones.

SEMANA 22: OPTIMIZACIÓN DE CONSULTAS Y RENDIMIENTO

Al hacer uso de la Base de Datos SQLite que está embebida en la memoria, se auditaron las peticiones de lectura repetitivas ejecutadas desde el backend de Node.js a través del método de ejecución de consultas.

Para mitigar la reducción del rendimiento que había en las consultas e implementar el crecimiento del volumen de registros históricos del diario de entrenamientos, se programó la creación de índices:

```
CREATE INDEX IF NOT EXISTS idx_usuarios_email ON usuarios(email);  
CREATE INDEX IF NOT EXISTS idx_logins_usuario_id ON logins(usuario_id);  
CREATE INDEX IF NOT EXISTS idx_rutinas_usuario_id ON rutinas(usuario_id);
```

Se agrega los siguientes índices sobre los campos de restricción que tienen unicidad en este caso el email y las claves foráneas usuario_id. A partir de esto, se redujo la complejidad a la hora de las búsquedas en tablas. Por tanto, se pasó a un escaneo por medio de índices haciéndolo mucho más rápido frente a llamadas concurrentes a los endpoints.

SEMANA 23: INTEGRACIÓN FINAL DE FUNCIONALIDADES

Se validó la comunicación asíncrona entre el cliente web y el backend local conectado a la base de datos. Para ello, se realizaron pruebas de extremo a extremo que verificaron que las llamadas realizadas con la API fetch manejaran correctamente los códigos de estado HTTP enviados por el servidor en server.js (200 para operaciones exitosas, 400 para errores en los parámetros y 404 para endpoints inexistentes). Además, se comprobó la correcta recepción y procesamiento de los datos JSON en la interfaz de usuario.

SEMANA 24: PRUEBAS DE USUARIO Y CALIDAD

En consonancia con los requisitos de seguridad y salud laboral para el puesto de Desarrollador de Software en prácticas, el equipo integró formalmente las directrices del plan de prevención de riesgos profesionales vinculándolas directamente a la fase de programación del proyecto:

1. **Identificación de Riesgos Ambientales y Físicos:** Durante las jornadas de desarrollo en HTML y Node.js, se evaluaron factores de riesgo físico como la fatiga visual por exposición continua a pantallas, la radiación menor de monitores y los deslumbramientos por iluminación artificial inadecuada en las salas de trabajo.
2. **Riesgos Ergonómicos y Psicosociales:** Se identificaron las malas posturas y el estatismo frente al ordenador que provocan dolores musculares o problemas en la espalda, así como la fatiga mental y el estrés derivado de la carga de trabajo que supuso reconstruir por completo la aplicación desde cero.
3. **Medidas Preventivas Técnicas y Organizativas:** Organización limpia del cableado del servidor y uso de sillas regulables para evitar riesgos de caídas al mismo nivel por tropiezos.
 - *Higiene Visual:* Establecimiento de pausas activas de trabajo, descansos visuales periódicos apartando la vista del monitor y control estricto de la iluminación para suprimir reflejos.
 - *Mitigación desde el Software:* Como medida técnica de prevención colectiva para paliar la fatiga visual de los atletas que utilicen la aplicación en salas interiores, la interfaz de `traincore_app.html` se desarrolló nativamente con un esquema de color adaptativo oscuro, disminuyendo de manera drástica la emisión de brillo de las pantallas.

SEMANAS 25 Y 26: REDACCIÓN DE LA MEMORIA FINAL Y REVISIÓN DE LA MEMORIA

Se recopilaron todos los apartados documentales generados durante el curso escolar para armar la memoria de proyecto unificada y corregir errores ortográficos. Se realizaron auditorías cruzadas simuladas en el aula con otros equipos de programación para verificar la claridad en la lectura del código fuente del backend y pulir pequeños fallos en las respuestas de control de excepciones.

SEMANAS 27 Y 28: PREPARACIÓN DE LA PRESENTACIÓN ORAL Y ENSAYO DE LA DEFENSA

Se realiza una reunión con el equipo para presentar nuestro proyecto final entre nosotros y realizar un ensayo, mirar errores, probar posibles incidencias en medio de la presentación por tanto se realizó un plan b en caso de incidentes como caída del internet, etc...

Por tanto el html si tiene un problema de internet sus bloques de control están dentro de un try catch. En caso de fallo de la interfaz con la conexión al servidor HTTP manda un error en el que dice no se puede conectar al servidor. Además, se restaura la interfaz de manera automática de manera visual, impidiendo que haya un congelamiento en medio de la presentación. También hay copias de la base de datos en caso tal de una eliminación accidental.

SEMANAS 29 Y 30: PRESENTACIÓN Y DEFENSA PÚBLICA Y EVALUACIÓN FINAL Y CIERRE

Presentación al docente y al aula de cómo funciona el proyecto, demostración en directo del sistema CRUD y realización de los cuestionarios finales de evaluación. Además, se limpió el repositorio de GitHub eliminando archivos innecesarios y se dejó preparada una versión final estable para la entrega.